AI-Architect Architecture Report

Project: Gemini Brownfield

Run ID: 20260828T135945Z-f88a464f

Status: Accepted — 2026-08-28 14:11 UTC

Final Review Verdict: PASS

Models used: gemini-3.1-flash-lite

Executive Summary

This architecture transforms the monolithic e-commerce platform into a scalable ecosystem, allowing individual modules (like payments or product catalog) to be updated and scaled independently. For users, this means a more responsive and reliable shopping experience, even during peak sales events. For operations and developers, it provides agility to deploy features without affecting the entire system, while ensuring adherence to GDPR compliance through better data isolation and encryption strategies.

Requirements & Constraints

Business Goal

Modernize the existing e-commerce monolith so it can scale during peak traffic, remain maintainable as the business grows, and evolve incrementally without disrupting ongoing operations.

Problem Statement

The current monolith has scalability and maintainability problems, especially during peak traffic. Changes are difficult to deploy safely, and tightly coupled functionality makes independent evolution difficult.

Users / Stakeholders

- Online customers
- customer support
- warehouse and inventory staff
- finance and payment operations
- product management
- software engineers
- operations staff

Functional Requirements

- User account management
- Product browsing and management
- Order processing
- Payment integration
- Product review functionality

Non-Functional Requirements

- Support up to 50,000 concurrent users during peak campaigns.
- Maintain high availability during peak periods.
- Keep customer-facing interactions responsive.

- Allow the system to scale as demand grows.
- Support incremental migration without major service disruption.

Cloud Provider
AWS

Budget
Cost efficiency should be considered.

Compliance Requirements

- GDPR compliance is required. Personal customer data must be protected and access to sensitive data should follow appropriate security controls.

Existing Systems

- Django backend
- React frontend
- Python based monolith

Recommended Architecture

Pattern
Strangler Fig migration to event-driven microservices

Rationale
The Strangler Fig pattern allows for the incremental modernization of the existing Django monolith into independent microservices. This approach mitigates risk by replacing functionality module-by-module rather than via a big-bang rewrite, ensuring high availability and business continuity during the transformation.

Components

ID	Name	Type	Purpose
COMP-001	Frontend Application	UI	Provides user interface and client-side logic.
COMP-002	API Gateway	Routing/Gateway	Routes requests between client and backend services.
COMP-003	Identity Service	Service	Manages users and profiles.
COMP-004	Product Service	Service	Product catalog and management.
COMP-005	Order Service	Service	Order processing.
COMP-006	Payment Service	Service	Payment processing.
COMP-007	Review Service	Service	Product feedback.
COMP-008	Event Bus	Infrastructure	Asynchronous communication.
COMP-009	Legacy Monolith	Legacy Service	Legacy system during migration.

COMP-001: Frontend Application

The React-based client-side application that interacts with the API Gateway.

Technology
React

Traces to features
FEAT-001, FEAT-002, FEAT-003, FEAT-006

Justified by
ADR-006

COMP-002: API Gateway

Central entry point that routes traffic to the legacy monolith or the new microservices based on domain.

Technology
AWS API Gateway

Traces to features
FEAT-004, FEAT-005, FEAT-006

Justified by
ADR-001, ADR-006

COMP-003: Identity Service

Handles authentication, registration, and user profile management.

Technology
Python/Django

Traces to features
FEAT-001

Justified by
ADR-002, ADR-004, ADR-005

COMP-004: Product Service

Handles product browsing and management capabilities.

Technology
Python/Django

Traces to features
FEAT-002

Justified by
ADR-002, ADR-004, ADR-006

COMP-005: Order Service

Manages customer orders and status updates.

Technology
Python/Django

Traces to features
FEAT-003

Justified by
ADR-002, ADR-003, ADR-004, ADR-005

COMP-006: Payment Service

Handles integration with external payment gateways.

Technology

Python/Django

Traces to features
FEAT-003

Justified by
ADR-002, ADR-003, ADR-004, ADR-005

COMP-007: Review Service

Manages customer reviews.

Technology
Python/Django

Traces to features
FEAT-002

Justified by
ADR-002, ADR-004

COMP-008: Event Bus

Message broker for inter-service communication.

Technology
AWS SNS/SQS

Traces to features
FEAT-004

Justified by
ADR-003

COMP-009: Legacy Monolith

The original monolithic application, being gradually replaced.

Technology
Python/Django

Traces to features
FEAT-005

Justified by
ADR-001

Data Flows

- Frontend Application → API Gateway
- API Gateway → Identity Service
- API Gateway → Product Service
- API Gateway → Order Service
- API Gateway → Legacy Monolith
- Order Service → Event Bus
- Event Bus → Payment Service

Architecture Decision Records

ADR-001: ADR-001: Strangler Fig Migration Pattern

Status: accepted

Context
The existing monolith is too tightly coupled to replace all at once without major disruption.

Decision
Use the Strangler Fig pattern to incrementally extract services.

Rationale
Minimizes risk of downtime and allows for iterative delivery of scalable services.

Alternatives Considered

- Big-bang rewrite
- Continued monolith maintenance

Positive Consequences

- Continuous delivery capability
- Lower risk of full-system failure

Negative Consequences / Trade-offs

- Increased complexity during transition period
- Need for routing layer management

Related Features: FEAT-005

Related Components: Legacy Monolith, API Gateway

Related Decision Topics: TOPIC-5

Evidence: KB-E010

ADR-002: ADR-002: Domain-Driven Service Boundaries

Status: accepted

Context
Need to decouple business capabilities to improve scalability and maintainability.

Decision
Use Domain-Driven Design (DDD) to define microservice boundaries based on business capabilities (Identity, Product, Order, Payment, Review).

Rationale
Aligns technical boundaries with business domains, reducing coupling and improving team autonomy.

Alternatives Considered

- Infrastructure-based decomposition
- Layer-based decomposition

Positive Consequences

- Improved service cohesion
- Clear ownership boundaries

Negative Consequences / Trade-offs

- Requires significant analysis of existing monolithic domains

Related Features: FEAT-001, FEAT-002, FEAT-003

Related Components: Identity Service, Product Service, Order Service, Payment Service, Review

Service

Related Decision Topics: TOPIC-1

Evidence: KB-E001

ADR-003: ADR-003: Async Event-Driven Communication

Status: accepted

Context
Synchronous request-response is a bottleneck and creates high coupling during peak load.

Decision
Adopt an asynchronous event-driven architecture for inter-service communication.

Rationale
Improves fault tolerance and system scalability under high concurrency by decoupling services.

Alternatives Considered

- Strict synchronous REST calls
- Shared database communication

Positive Consequences

- Enhanced availability
- Resilience to downstream service failure

Negative Consequences / Trade-offs

- Increased complexity in distributed data consistency
- Eventual consistency challenges

Related Features: FEAT-004, FEAT-006

Related Components: Event Bus, Order Service, Payment Service

Related Decision Topics: TOPIC-3

Evidence: KB-E002, KB-E005

ADR-004: ADR-004: Database-per-Service

Status: accepted

Context
Monolithic database is a single point of failure and bottleneck.

Decision
Each microservice must own its private data store (database-per-service).

Rationale
Prevents coupling via database, allowing independent schema evolution and scaling.

Alternatives Considered

- Shared database with different schemas
- Single large database

Positive Consequences

- Improved performance isolation
- Ability to select optimal DB technology per service

Negative Consequences / Trade-offs

- Requires more complex data aggregation for reporting
- Cross-service transactions require saga patterns

Related Features: FEAT-004

Related Components: Identity Service, Product Service, Order Service, Payment Service, Review Service

Related Decision Topics: TOPIC-2

Evidence: KB-E004, KB-E009

ADR-005: ADR-005: Security and GDPR Compliance Architecture

Status: accepted

Context
GDPR requires strict protection of customer PII and adherence to security controls.

Decision
Implement PII encryption at rest and in transit; centralized identity management for access control; audit logging for data access.

Rationale
Essential for meeting regulatory compliance and safeguarding customer trust.

Alternatives Considered

- Per-service custom auth
- No explicit encryption

Positive Consequences

- Compliance with GDPR
- Improved data security posture

Negative Consequences / Trade-offs

- Increased key management complexity

Related Features: FEAT-001

Related Components: Identity Service, Order Service, Payment Service

Related Decision Topics: TOPIC-7

Evidence: KB-E003, KB-E002

ADR-006: ADR-006: Scaling and Availability Strategy

Status: accepted

Context
System must support 50,000 concurrent users with high availability and responsiveness.

Decision
Deploy auto-scaling for compute services, use CDN for static assets, and implement robust caching (Redis) for product and session data.

Rationale
Ensures responsiveness and elastic scaling under peak load conditions.

Alternatives Considered

- Manual scaling
- Single large server

Positive Consequences

- Supports high concurrency
- Improved user-facing responsiveness

Negative Consequences / Trade-offs

- Increased infrastructure cost
- Cache consistency management requirements

Related Features: FEAT-004, FEAT-006

Related Components: Frontend Application, API Gateway, Product Service

Related Decision Topics: TOPIC-4

Evidence: KB-E002, KB-E006, KB-E010

Migration Plan

Step 1: Introduce API Gateway and Event Bus

Objective

Establish the routing and communication foundation.

Changes

- Deploy AWS API Gateway
- Deploy message broker/Event Bus (e.g., Amazon EventBridge or SQS/SNS)

Coexistence / Data Strategy

The API Gateway routes all requests to the legacy monolith.

Exit Condition

All traffic flows through the API Gateway.

Step 2: Extract Identity Service

Objective

Move user account management to an independent service to enable centralized security.

Changes

- Create Identity Service (Python/FastAPI or Django)
- Migrate user tables to service-specific database

Coexistence / Data Strategy

New logins go through Identity Service; existing sessions stay in Legacy Monolith; JWT handles token validation across services.

Exit Condition

User authentication is decoupled from the monolith.

Step 3: Extract Product Service

Objective

Scale product browsing independently for peak traffic.

Changes

- Extract Product logic to Product Service
- Deploy read-replica database for product catalog

Coexistence / Data Strategy

API Gateway routes product catalog requests to the new service; internal calls are redirected.

Exit Condition

Product catalog is served by Product Service.

Step 4: Extract Order, Payment, and Review Services

Objective

Finalize decomposition to decouple transactional flows.

Changes

- Extract Order, Payment, and Review modules
- Implement Sagas for distributed transactions

Coexistence / Data Strategy

Order processing triggers events via Event Bus; Legacy Monolith is retired for these domains.

Exit Condition

All core modules exist as independent services.

Evidence / Literature

Curated Knowledge Base Evidence

ID	Source	Page	Excerpt
KB-E001	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	The reviewed studies reveal several distinct approaches to monolith decomposition, each with specific advantages for e-commerce contexts. **a. Domain-Driven Design (DDD) Approaches:** Multiple studies emphasize Domain-Driven Design as a foundational approach for identifying service boundaries in e-commerce systems. A...
KB-E002	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	Microservices architecture has emerged as a compelling alternative, offering the promise of enhanced scalability, improved fault tolerance, and greater development agility [1]. By decomposing monolithic applications into smaller, independently deployable services, organizations can achieve better resource utilization,...
KB-E003	Rag Database/box2_domain/ecommerce_microservices_challenges_ibrahim_luong.md	0	ensuring the security of microservices and their interactions is crucial in e-commerce, where sensitive customer data is often processed. Abgaz et al. [6] have also stated the importance of strong security and identity management inside a microservices ecosystem. Scalability is another challenge, as e-commerce platfor...
KB-E004	Rag Database/box1_patterns/architecture_patterns_v2.md	0	**Solution mechanics:** Each service exclusively owns its data; persistent data is accessed only through that service's API, never by another service reaching into its tables. Implementation variants (increasing isolation): private-tables-per-service, schema-per-service, and database-server-per-service (private-tables...

ID	Source	Page	Excerpt
KB-E005	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	**b. High Load Scenarios:** Asynchronous patterns (message queues) demonstrate superior throughput and availability [4]. **c. Availability:** Event-driven architectures provide better fault tolerance and system availability under stress [4]. These findings suggest that e-commerce systems should employ hybrid approac…
KB-E006	Rag Database/box2_domain/ecommerce_microservices_challenges_ibrahim_luong.md	0	## 2.2 Monolithic and Microservices Architectures Comparison *(printed p. 478)* In the e-commerce landscape, choosing between microservices and traditional monolithic architectures requires careful consideration of various factors. Monolithic architectures, characterized by their single, tightly coupled codebase. How…
KB-E007	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	# Monolith-to-Microservices-Migration im E-Commerce: Zerlegung, Event-Driven Patterns und Leistung (kuratorischer Auszug) - **Source title:** Migrating Monolithic E-Commerce Systems to Microservices: A Systematic Review of Event-Driven Architecture Approaches (curated excerpts) - **Author(s):** Stephen W. Bulus, Olub…
KB-E008	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	## Introduction — relevante Absätze zur E-Commerce-Migration *(printed p. 2)* The rapid evolution of e-commerce platforms has necessitated architectural paradigm shifts to meet increasing demands for scalability, reliability, and agility [2]. Traditional monolithic architectures, while providing simplicity in develop…
KB-E009	Rag Database/box2_domain/ecommerce_polyglot_persistence_microsoft.md	0	With a domain-driven microservices approach, each service uses the database that fits its data characteristics. Each microservice owns its private data store. This design prevents unintentional coupling between services and supports independent updates and deployments without coordinating changes across the system. #…
KB-E010	Rag Database/box1_patterns/microservices-on-aws.pdf	5	Modernizing to microservices Microservices are essentially small, independent units that make up an application. Transitioning from traditional monolithic structures to microservices can follow various strategies. Are you Well-Architected? 2

Validation & Reviewer Findings

Overall Verdict: PASS

Refinement rounds: 1

No open findings were recorded on the accepted review.

Traceability

Features -> Decisions / Components

Feature	Name	ADRs	Components
FEAT-001	User Account Management	ADR-002, ADR-005	COMP-001, COMP-003
FEAT-002	Product Catalog and Review System	ADR-002	COMP-001, COMP-004, COMP-007
FEAT-003	Checkout and Payment Integration	ADR-002	COMP-001, COMP-005, COMP-006
FEAT-004	Elastic Scalability Infrastructure	ADR-003, ADR-004, ADR-006	COMP-002, COMP-008
FEAT-005	Incremental Migration Framework	ADR-001	COMP-002, COMP-009
FEAT-006	Interaction Responsiveness Optimization	ADR-003, ADR-006	COMP-001, COMP-002

Decision Topics -> ADRs

Topic	Question	ADRs
TOPIC-1	service decomposition and boundaries	ADR-002
TOPIC-2	data ownership and persistence strategy	ADR-004
TOPIC-3	integration style: synchronous vs asynchronous communication	ADR-003
TOPIC-4	scaling and availability strategy	ADR-006
TOPIC-5	brownfield migration and evolution strategy	ADR-001
TOPIC-6	technology conservation vs replacement	—
TOPIC-7	security and compliance architecture	ADR-005

ADRs -> Evidence

ADR	Evidence
ADR-001	KB-E010
ADR-002	KB-E001
ADR-003	KB-E002, KB-E005
ADR-004	KB-E004, KB-E009
ADR-005	KB-E003, KB-E002
ADR-006	KB-E002, KB-E006, KB-E010

Limitations / Open Items

No unresolved items are recorded on this run.